COGS 188 Platformer Project Proposal

Project Proposal: Which Course-Taught AI Methods Can Solve a Procedural Platformer?

Team

Team members: Add the project team's names.

Project type: AI solve-a-problem implementation project.

Public repository: ManjuDaBeast/COGS188_Platformer

Abstract

This project will analyze and compare representative methods covered in the course lecture materials on a seed-deterministic procedural platformer. The task requires selecting timed movement, jump, and glide actions while navigating gaps, changing elevations, moving platforms, hazards, and enemies. The existing Gymnasium environment and simple random and hand-written baselines provide a working computational problem. We will compare uninformed search (breadth-first, depth-first, and uniform-cost search), heuristic search (greedy best-first and A*), discrete dynamic programming (value and policy iteration), Monte Carlo control, local search (hill climbing and simulated annealing), and a state-independent UCB bandit baseline. Since these methods differ in whether they learn from episodes or plan with a model, results will be reported in separate access tracks and compared using held-out completion, progress, and computational cost. We hypothesize that model-based search will do well when planning on a known level, Monte Carlo control may better transfer a learned policy to new levels, and a state-independent bandit will be a weak full-game controller. These are hypotheses; the benchmark will determine the empirical ranking.

Background and Motivation

The project environment is a sequential decision problem: actions change the agent's position and velocity, platforms and enemies may move, and a mistake can prevent reaching the goal. The environment is already implemented as a Gymnasium task with

deterministic level generation for a given seed, a 42-dimensional normalized observation, eight discrete actions, and easy, medium, and hard level tiers. There are also random and hand-written goal-directed policies, seed-paired evaluation, and replay visualization.

The initial experiments show that the task is not solved well by simple behavior alone: on the earlier shared-level benchmark, random actions completed 0% of episodes and a hand-written heuristic completed 12.7% overall, including 6% on hard levels. Those numbers came from exploratory project runs, not from the proposed course-method comparison; they motivate the benchmark but will not be used to rank the new methods. The repository also contains implementations and results for methods outside the supplied lecture set. Those methods are excluded from this proposal's comparison and from its conclusions.

Why the task is interesting

The task requires planning over long action sequences with eight possible actions at each step. An exhaustive search to the environment's 300-step evaluation limit could inspect (8^{300}) action sequences. Exact search or dynamic programming on the raw continuous physics state is not practical. Discretizing the state makes tabular methods possible but introduces a resolution trade-off: coarse bins merge states that need different actions, while fine bins create many rarely visited states. The generator also changes level geometry and obstacle placement across seeds, so a method that plans well on one level may not transfer to another.

Research question

Among representative methods taught in the supplied search, local search, bandit, dynamic-programming, and Monte Carlo lectures, which approaches achieve the best held-out platformer completion for their available information and computational budget, and which transfer best to unseen procedural levels?

Hypotheses

- **H1:** A* or uniform-cost planning on a level-specific abstract platform graph will achieve strong completion when given the current level model, because the goal and level connectivity can guide search.
- **H2:** Monte Carlo control on a discretized observation may have lower performance on individual familiar layouts than a planner, but will be more directly reusable on new level seeds after training.
- **H3:** Value or policy iteration will be useful on a small abstract MDP, but performance will depend on transition-model accuracy and state discretization.
- **H4:** A bandit that selects actions without conditioning on state will perform poorly as a full controller, because platforming actions have state-dependent and delayed consequences.
- **H5:** Local search will improve short-horizon decisions in some states, but may be sensitive to horizon, objective, and local optima.

These are testable predictions, not reported findings. The study will allow the data to contradict them.

Problem Statement

Given the current environment observation and one generated level, select actions from the eight-action set to reach the goal before death or the episode time limit. Evaluate both:

1. **Per-level planning:** a method may inspect the current level and, where specified, query a transition simulator before or during an episode.
2. **Policy transfer:** a method learns or constructs a policy and is then evaluated on level seeds not used to fit or tune it.

The primary outcome is the proportion of held-out episodes that reach the goal. The problem is measurable and repeatable: the environment produces seeded levels and records completion, return, progress, gems, episode length, stall fraction, and death cause.

Data and Environment

No human or external data are used. Training and test data are interactions with the procedural simulator.

- **State and actions:** normalized 42-element observation and eight discrete actions (no-op, left, right, jump, left+jump, right+jump, glide, and right+glide).
- **Difficulty:** easy, medium, and hard tiers, with progressively wider gaps, narrower platforms, greater vertical variation, and more moving platforms and obstacles.
- **Course-compatible baseline policies:** random action and the existing hand-written goal-directed policy; these are references, not learned algorithms in the method comparison.
- **Training seeds:** proposed range 2000–2099 for Monte Carlo episodes, model/abstraction construction, and pilot work.
- **Validation seeds:** proposed range 2100–2149 for choices such as discretization, heuristic weights, and local-search horizon.
- **Held-out seeds:** proposed range 2200–2249, evaluated at all three difficulties for paired tier comparisons.
- **OOD seeds:** proposed range 2250–2299, evaluated at medium difficulty under both standard and moving-heavy layout variants.

Before any run, inspect existing artifacts and confirm that the proposed seeds have not already been consumed. Freeze the held-out list before tuning. Record exact seeds, settings, and per-episode outputs in JSON.

Methods to Compare

The provided lecture PDFs cover different kinds of approaches. An MDP is the problem formulation, while stochasticity and observability describe the environment; they are not competing algorithms. The study will implement representatives of each computational method family below, explicitly analyze the other named variants, and explain any abstraction required.

1. Uninformed search: breadth-first, depth-first, and uniform-cost search

Use breadth-first search (BFS) and uniform-cost search (UCS) on the abstract platform-transition graph. BFS minimizes the number of platform transitions. UCS minimizes an explicit nonnegative edge cost that includes simulated transition time and a fixed penalty for collision risk. DFS will be analyzed as a memory-light but non-optimal search and included as a bounded baseline if repeated-state handling prevents cycles. Search will be limited by a shared node-expansion budget. These methods cannot search the raw continuous state tree to full episode depth.

2. Heuristic search: A*

Use A* on the same abstract graph and edge costs as UCS. The initial heuristic will estimate remaining cost from horizontal distance to the goal divided by a validation-measured optimistic progress-per-transition. The heuristic is fixed before held-out evaluation. Greedy best-first search using only this heuristic will also be compared to show the effect of ignoring path cost. A* is a planner that may use level geometry; it is not a learned, reusable action policy. Report its access and planning cost separately from interaction-only methods.

3. Dynamic programming: value iteration and policy iteration

Construct a finite abstract MDP whose states encode the current platform, coarse movement mode, and nearby threat/landing category. Estimate transition probabilities and expected rewards by repeated simulator transitions for each abstract state-action pair. Run value iteration and policy iteration on the same estimated model. Evaluate both the resulting policy and the cost of building the model. The report will state that this is approximate DP on an abstract model, not exact DP on the original continuous environment.

4. Monte Carlo control

Implement an on-policy Monte Carlo control method over a documented discretization of selected observation features. Use complete episode returns to estimate state-action values and an epsilon-soft policy for exploration. Compare first-visit and every-visit updates only if the predeclared compute budget permits; otherwise use first-visit as the primary implementation and describe every-visit as a planned sensitivity analysis. Train on training seeds and freeze the policy before held-out evaluation.

5. Local search: hill climbing and simulated annealing

For a known current state and level, search over a bounded sequence of future actions using simulator rollouts. Hill climbing accepts improving perturbations; simulated annealing may accept worse intermediate sequences according to a fixed cooling schedule. Execute only the first action, then replan from the new state. This is model-based control and will be reported in the planning track with its simulator-query budget; it will not be presented as a model-free learned policy.

6. Bandit baseline: UCB

Implement a standard UCB action selector with one reward estimate per action, without conditioning on the platformer state. This provides a deliberately limited course-covered baseline illustrating the bandit assumption. Do not silently convert it to a contextual bandit, since that is a different method and is not in the supplied lecture set.

Other lecture concepts and variants

- **MDP formulation, stochastic transitions, and partial observability:** define the problem and its information limits; they are used to explain assumptions and are not scored as algorithms.
- **Exact dynamic programming on the full environment:** not feasible over the continuous physics state. The comparison uses only the described finite abstract MDP.
- **Monte Carlo exploring starts and policy evaluation:** discuss how these relate to state coverage and evaluation. Use an epsilon-soft control policy because arbitrary start-state sampling may not be available through the environment reset interface.
- **Local-search variants:** compare first-choice/random-restart hill climbing with simulated annealing if the fixed rollout budget permits; otherwise identify the variant not run as a limitation.

Abstraction and fairness

Search and DP need a finite planning representation, while Monte Carlo control needs a manageable tabular representation. Define and test these representations before the pilot. Use the same abstraction where it is meaningful, then report its resolution and aliasing risks. Local search and graph search can query the simulator; Monte Carlo control learns from interaction; UCB does not model state transitions. Because information access differs, report two scorecards rather than a misleading single ranking:

- **Interaction/transfer track:** Monte Carlo control, UCB, random policy, and hand-written baseline, evaluated on fresh levels after training or policy construction.
- **Per-level planning track:** BFS, UCS, A*, value/policy iteration with their model-construction cost, and replanning local search, evaluated with explicit simulator-query and wall-clock budgets.

All methods use the same held-out level seeds within their applicable track. Report environment transitions, planner expansions/simulator calls, model construction cost, wall time, and memory where feasible. Best individual replays are illustrative and do not determine the ranking.

Experimental Procedure

1. Audit and simplify the existing environment interface without changing its physics or tier definitions.
2. Add unit tests for abstract state construction, graph transitions, cost nonnegativity, DP transition normalization, Monte Carlo updates, UCB action accounting, and search-budget enforcement.
3. Use a small pilot on validation seeds to select one documented discretization, heuristic, edge-cost rule, and local-search budget.

4. Freeze all choices and evaluate every method on the paired held-out seeds.
5. Run the separate moving-heavy evaluation only after the main protocol is locked.
6. Publish code, exact run configuration, aggregate and per-seed metrics, limitations, and representative replays.

No held-out results will be used to tune hyperparameters or abstractions. The existing out-of-scope algorithm outputs will not be incorporated into the new course-method leaderboard.

Evaluation Metrics

For (N) episodes with completion indicator (C_i), completion rate is

[= _{i=1}^{N} C_i.]

Primary evaluation reports completion separately for easy, medium, hard, and overall held-out episodes. Also report mean maximum progress, mean episodic return, gem fraction, stall fraction, deaths by cause, and episode length. For planners, report expansions and simulator calls per action or episode. For all methods, report wall-clock time and memory where practical. For OOD evaluation, report the moving-heavy minus standard completion-rate gap on the same seeds.

Because methods may use different numbers of simulator calls and have different model access, show completion as a function of both environment interactions and wall-clock/planning budget. Report per-seed values and uncertainty intervals; do not treat episodes from one trained policy as independent training replications.

Expected Results and Success Criteria

The project succeeds by producing a reproducible comparison and explaining the trade-offs, even if the methods fail to complete many hard episodes. The main scientific conclusion will identify which methods do best in the planning and interaction/transfer tracks, and how much computation or model information each used. The planned method matrix includes BFS, DFS, UCS, greedy best-first, A*, value iteration, policy iteration, Monte Carlo control, UCB, hill climbing, and simulated annealing. Any variant omitted under the final compute budget will be identified explicitly rather than represented by another method's result.

Before results exist, the expected ordering is only a hypothesis: A* may benefit from the known, goal-directed geometry in per-level planning; Monte Carlo control may be more reusable across seeds but is vulnerable to coarse state bins and sparse terminal success; UCS may be competitive when edge costs reflect risk; DP may be effective only if its abstract transition model is sufficiently accurate; local search may be horizon-sensitive; and state-independent UCB is expected to be weak. The empirical comparison—not this expectation—will answer the research question.

Ethics, Privacy, and Limitations

This project uses synthetic simulator data and does not collect personal information. Important risks are methodological: discretization can hide important state differences; a hand-designed heuristic or cost can encode unfair prior knowledge; planner methods may receive more simulator information; and tuning can leak held-out levels into development. Address these by documenting abstractions, separating access tracks, pre-freezing held-out seeds, reporting compute and model access, and retaining negative results.

The environment is a simplified game and results do not imply real-world robot or human safety. The abstract graph and finite MDP are approximations, not the original continuous dynamics. Any conclusion will be scoped to this environment and the tested budgets.

Team Expectations

- Assign owners for algorithm implementation, environment/abstraction tests, and experiment tracking/reporting; review one another's changes.
- Keep method assumptions, simulator access, budgets, and seed lists in the repository.
- Do not change the confirmatory test protocol without recording the reason and affected results.
- Share blockers and unexpected failures early; use tests and logged outputs to resolve disagreements.
- Agree on milestone dates and team responsibilities at project start.

Proposed Timeline

Week	Work before check-in	Deliverable / decision
1	Verify the course method list, repository history, and unused seed ranges.	Freeze the course-only scope and define the planning/learning scorecards.
2	Design and test the finite abstractions and simulator-transition interface.	Document abstraction accuracy and limits before algorithm work.
3	Implement BFS, UCS, A*, and local search with fixed expansion/query budgets.	Pass correctness tests and run validation-only smoke tests.
4	Implement abstract-MDP value/policy iteration, Monte Carlo control, and UCB.	Verify update equations and run validation-only smoke tests.
5	Select settings on validation seeds, lock the protocol, and run held-out tests.	Produce paired per-seed scorecards with compute costs.
6	Run OOD tests, analyze failure modes, and prepare	Deliver the course-aligned report and reproducibility

Week	Work before check-in	Deliverable / decision
	reports/replays.	instructions.

Repository and References

Existing project materials

- Public repository: https://github.com/ManjuDaBeast/COGS188_Platformer
- Environment source
- Course-method lecture PDFs

Course lecture sources

The methods in this proposal are restricted to the supplied course materials:

1. *L02 - Uninformed search*: breadth-first, depth-first, and uniform-cost search.
2. *L03 - Heuristic search*: greedy best-first and A*.
3. *L04 - Local search*: hill climbing and simulated annealing.
4. *L05 - Stochasticity and Observability*: stochastic transitions and partial-observation problem characteristics.
5. *L08 - Bandits*: UCB and bandit action selection.
6. *L09-10 MDP*: Markov decision process formulation.
7. *L11 DynamicProgramming*: value iteration and policy iteration.
8. *L13-14 MonteCarlo*: Monte Carlo policy evaluation and control, including first-visit/every-visit and exploring-start concepts.

Background literature

1. Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press. http://incompleteideas.net/book/the-book-2nd.html
2. Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson. Search, local search, MDP, and planning chapters.
3. Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). “Finite-time Analysis of the Multiarmed Bandit Problem.” *Machine Learning*, 47, 235–256. https://doi.org/10.1023/A:1013689704352